CM1 : patrons de conception essentiels

1 Le patron Composite

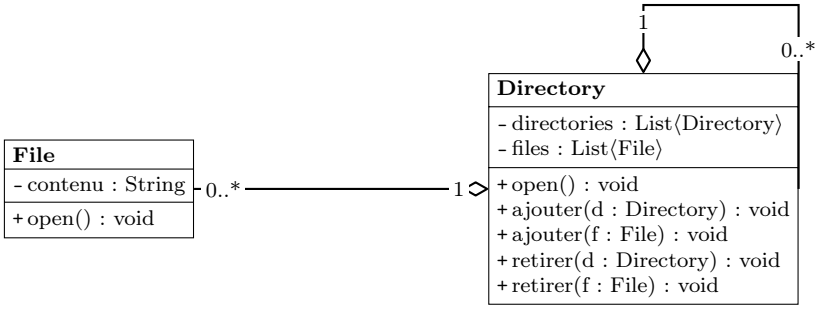
1.1 Les bases

Le problème

Considérons un système de gestion de fichiers et de dossiers. Il suit les règles suivantes :

1. les fichiers et les dossiers peuvent s'ouvrir ;
2. les dossiers peuvent contenir n'importe quoi, et les fichiers ne peuvent rien contenir (à part du texte, en attribut).

Un diagramme UML correct pour ce problème est le suivant.

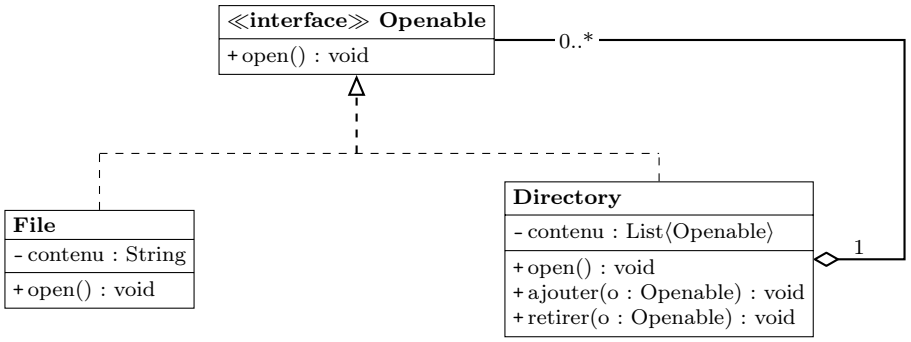


Ce diagramme pose deux problèmes.

— Pour ajouter un nouveau type d'objet dans un dossier (des images, par exemple), il faudra créer une nouvelle liste.

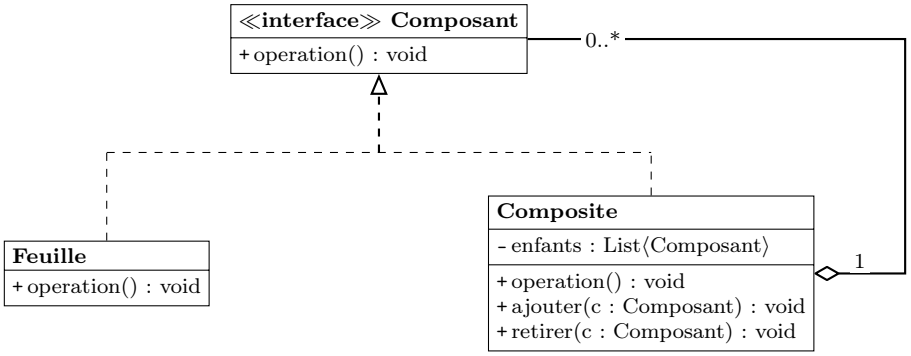
— `File` est utilisé directement dans `Directory`, alors qu'il faut privilégier les attributs de type abstrait.

Pour les résoudre, il suffit de constater qu'il existe une relation entre ces deux classes qui n'a pas été utilisée : elles sont toutes deux *ouvrables*. En prenant cela en compte, nous obtenons le diagramme suivant.



Ceci est un exemple d'utilisation du patron Composite.

Le schéma classique du patron Composite



Le patron fait intervenir trois rôles.

— Le **composant** est l'abstraction commune. Il déclare les opérations que le client pourra demander, sans distinguer un élément simple d'un élément composé.

— La **feuille** est un composant élémentaire : elle réalise l'opération pour elle-même et ne contient rien.

— Le **composite** est un composant qui contient d'autres composants. Il réalise l'opération en la déléguant à ses enfants, puis en combinant les résultats s'il y a lieu.

Le point essentiel est que le composite contient des *composants*, et non des feuilles : c'est ce qui permet à un composite d'en contenir un autre, et donc à la structure d'être un arbre de profondeur quelconque.

À retenir

Les conditions d'utilisation du patron Composite.

— Les objets du domaine s'organisent en une structure arborescente : un objet de type `A` peut contenir un objet de type `A`.

— Il existe une opération qui a un sens aux deux niveaux, et dont le résultat sur un composite se déduit de ceux de ses enfants.

Les avantages de ce patron.

— Le client traite de la même manière un objet simple et un objet composé : il n'a pas à savoir à quoi il s'adresse, donc ni test de type, ni distinction de cas.

— Ajouter une nouvelle sorte de composant ne modifie aucune classe existante (principe ouvert/fermé).

— Le composite dépend d'une abstraction et non d'une classe concrète (principe d'inversion des dépendances).

— La récursion de la structure se traduit par une récursion des traitements, qui s'écrivent une fois pour toutes.

Son prix.

— L'interface commune ne décrit vraiment que les méthodes partagées par tous les éléments de la hiérarchie.

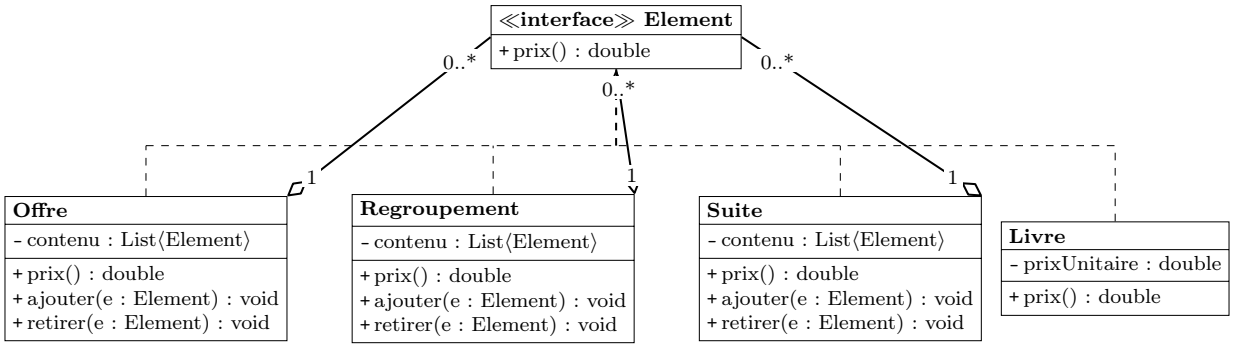
— Le patron ne permet pas, en lui-même, de restreindre ce qu'un composite accepte comme enfant.

1.2 Emboîtement de composites

Le problème

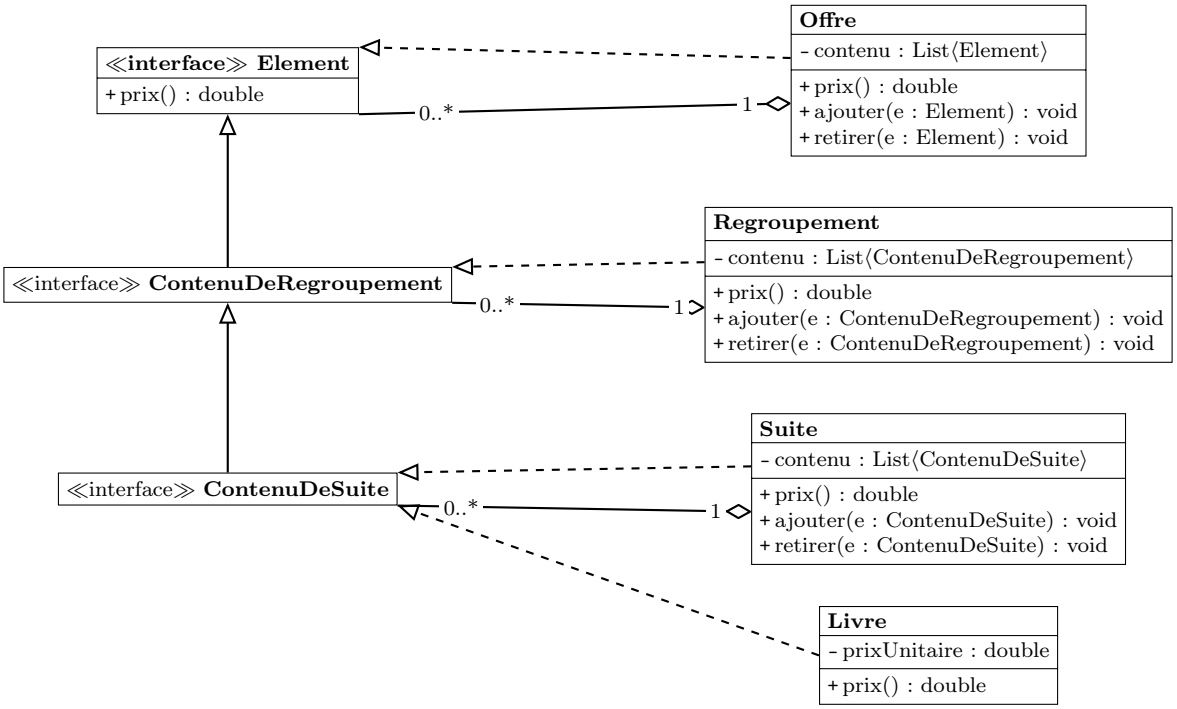
Reprenons le catalogue d'un éditeur. Il est organisé en quatre niveaux : les *offres* (niveau 0), les *regroupements* (niveau 1), les *suites* (niveau 2) et les *ouvrages* (niveau 3). Un élément ne peut contenir que des éléments de niveau supérieur ou égal au sien : une offre peut donc tout contenir, un regroupement peut contenir des regroupements, des suites et des ouvrages, et une suite ne contient que des suites et des ouvrages.

Toutes les classes partagent une même opération, le calcul du prix. Le patron Composite s'applique donc, et donne le diagramme suivant.



Ce diagramme est conforme au patron, mais il ne dit rien de la règle des niveaux. Comme les trois composites contiennent des `Element`, rien n'empêche d'ajouter une offre à une suite. La règle devrait donc être vérifiée à l'exécution, dans chaque méthode `ajouter`, ce qui soulève trois objections : le modèle n'exprime plus une contrainte pourtant structurelle, l'erreur n'est détectée qu'à l'exécution, et chaque `ajouter` contient un test de type.

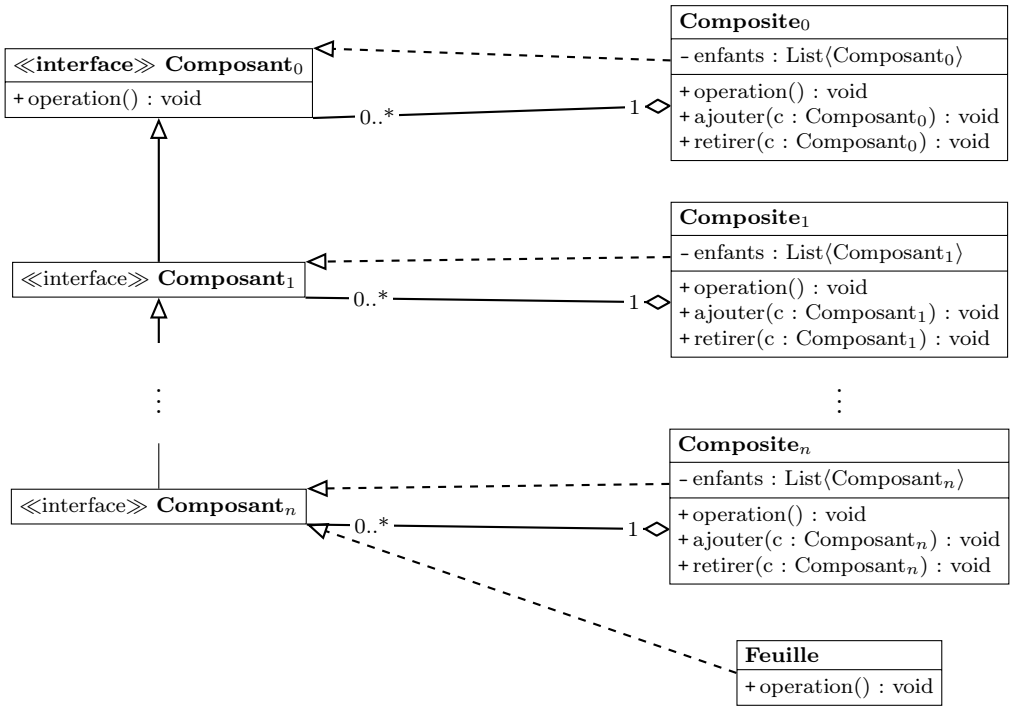
Pour le résoudre, il suffit de constater que « être de niveau au moins `i` » est une propriété de plus en plus exigeante à mesure que `i` augmente : un ouvrage est de niveau au moins 2, donc aussi de niveau au moins 1. Ces propriétés s'emboîtent, et une interface peut en étendre une autre. Nous obtenons le diagramme suivant.



Chaque interface décrit ce qu'un composite accepte comme enfant, et l'héritage entre interfaces traduit l'emboîtement des niveaux : comme `ContenuDeSuite` étend `ContenuDeRegroupement`, tout ce qu'une suite accepte, un regroupement l'accepte aussi. Il n'y a qu'une interface par composite :

les feuilles implémentent la dernière de la chaîne, celle du composite le plus profond. La règle des niveaux n'est plus vérifiée nulle part : elle est portée par le typage, donc garantie à la compilation.

Le schéma général



Le composite de niveau `i` contient des $\texttt{Composant}_i$, et implémente $\texttt{Composant}_i$: il peut donc se contenir lui-même, et être contenu par tous les niveaux au-dessus de lui. L'agrégation est la réciproque de l'implémentation : le composite réalise l'interface qu'il collectionne. Les feuilles implémentent $\texttt{Composant}_n$, la dernière interface de la chaîne.

À retenir

Les conditions d'utilisation de cet emboîtement.

— On est dans les conditions d'utilisation du patron Composite, mais les composites ne sont pas interchangeables : chacun n'accepte qu'une partie des composants.

— Ces restrictions sont emboîtées : ce qu'accepte un composite est inclus dans ce qu'accepte le composite du niveau au-dessus. Sans cette propriété, la chaîne d'interfaces n'existe pas.

Les avantages de cet emboîtement.

— La contrainte de contenu est portée par les types : elle est garantie à la compilation, et aucune méthode ne contient de test.

— Le modèle dit la règle, au lieu de la cacher dans le corps des méthodes.

— Ajouter un niveau intermédiaire se fait en insérant une interface dans la chaîne et en écrivant son composite ; les autres niveaux ne changent pas.

Son prix.

— Le nombre d'interfaces croît avec le nombre de niveaux, pour des interfaces dont la plupart ne déclarent rien de nouveau.

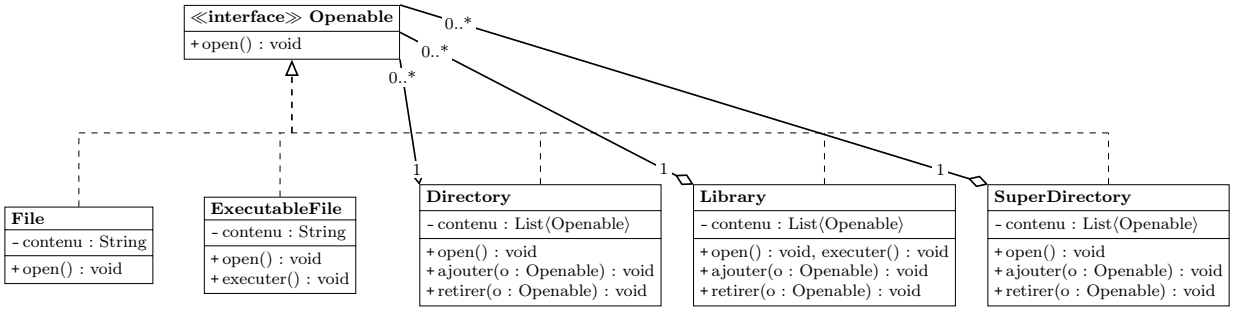
— L'uniformité du client est perdue en partie : le code qui construit le catalogue doit connaître le niveau de ce qu'il manipule. Celui qui se contente d'appeler l'opération commune ne voit, lui, aucune différence.

1.3 Fusion de composites

Reprenons le système de gestion de fichiers, enrichi. On dispose maintenant de plusieurs types d'objets :

1. les fichiers exécutables ;
2. les fichiers non exécutables ;
3. les dossiers, qui peuvent contenir des dossiers et n'importe quel type de fichier ;
4. les bibliothèques, qui sont des dossiers ne contenant que des bibliothèques ou des fichiers exécutables ;
5. les super-dossiers, qui ne contiennent que des dossiers ou des bibliothèques.

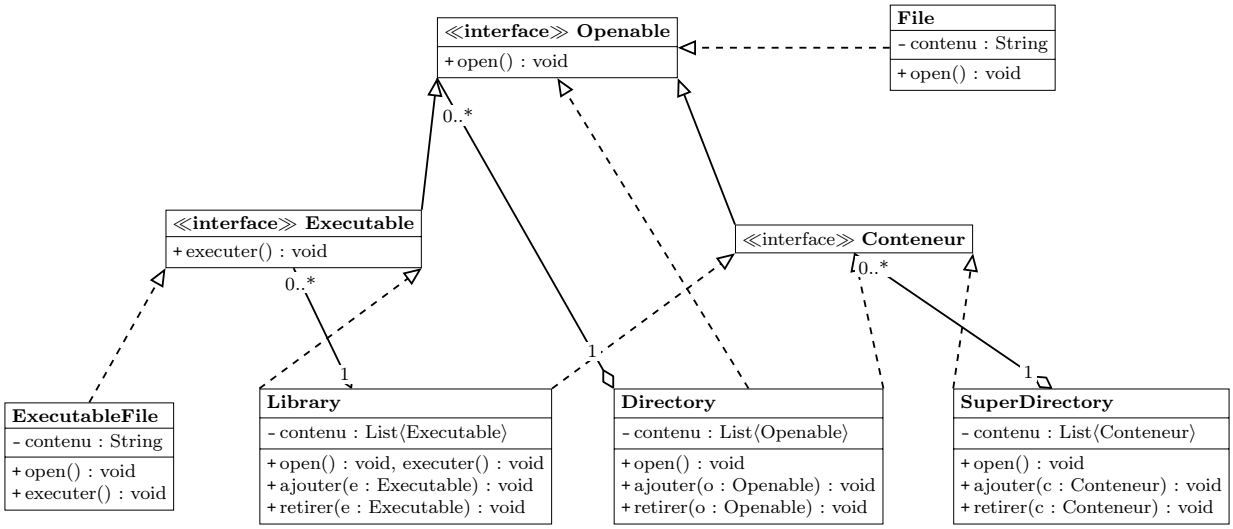
Les cinq classes partagent une même opération, l'ouverture. En appliquant directement le patron Composite, avec une seule interface, on obtient le diagramme suivant.



Ce diagramme ne dit rien des règles 4 et 5 : comme les trois composites contiennent des `Openable`, rien n'empêche de ranger un dossier dans une bibliothèque, ni un fichier dans un super-dossier. Il faudrait donc coder les restrictions dans les méthodes d'ajout, ce qui peut enfreindre le principe ouvert/fermé si cela passe par un test de type.

Remarquons qu'il y a trois composites (le dossier, la bibliothèque et le super-dossier) et que chacun restreint ce qu'il accepte. Mais, contrairement à la section précédente, ces restrictions ne s'emboîtent pas. Une bibliothèque n'accepte que des exécutables, un super-dossier n'accepte que des conteneurs, et aucun de ces deux ensembles ne contient l'autre : un fichier exécutable n'est pas un conteneur, un dossier n'est pas un exécutable. La chaîne d'interfaces de la section 1.2 ne s'applique donc pas.

En revanche, une bibliothèque appartient aux deux familles : c'est un conteneur, puisqu'elle contient, et c'est un exécutable, puisqu'on peut l'exécuter. Il suffit alors de donner une interface à chaque famille et de laisser la bibliothèque réaliser les deux.



Chaque interface nomme une famille de contenu : `Executable` désigne ce qu'une bibliothèque accepte, `Conteneur` ce qu'un super-dossier accepte, et `Openable` ce qu'un dossier accepte, c'est-à-

dire tout. Une classe qui appartient à plusieurs familles réalise plusieurs interfaces : c'est le cas de `Library`, qui est à la fois un exécutable et un conteneur, et c'est ce qui lui permet d'être rangée aussi bien dans une bibliothèque que dans un super-dossier.

Comme en section 1.2, les règles de contenu sont portées par le typage : il est impossible d'ajouter un dossier à une bibliothèque, puisque `Directory` ne réalise pas `Executable`. La différence est que les interfaces ne forment plus une chaîne mais un treillis, dont il faut dessiner la forme au cas par cas.

À retenir

Quand les restrictions ne s'emboîtent pas.

— On fabrique une interface par *famille de contenu*, c'est-à-dire par composite qui restreint ce qu'il accepte.

— Une classe qui appartient à plusieurs familles réalise plusieurs de ces interfaces : c'est la fusion.

— Il n'y a pas de schéma type : la forme du diagramme reproduit celle des règles de contenu, et doit être établie à chaque fois.

La méthode.

1. Lister les composites, et pour chacun l'ensemble des classes qu'il accepte.
2. Comparer ces ensembles. S'ils s'emboîtent, c'est une chaîne (section 1.2) ; sinon, une interface par ensemble.
3. Faire réaliser à chaque classe toutes les interfaces dont elle est membre.

Son prix.

— Le nombre d'interfaces suit le nombre de composites, et leurs relations sont à vérifier une par une.

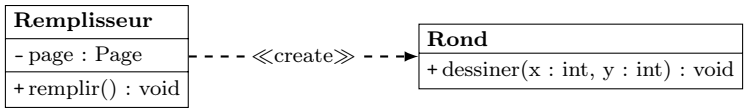
— Une classe peut réaliser plusieurs interfaces, mais une seule implémentation : si deux familles demandaient le même nom de méthode avec deux comportements différents, il faudrait trancher.

2 Le patron Fabrique

2.1 Les bases

Le problème

On veut remplir une page de formes. Une classe `Remplisseur` s'en charge : elle parcourt la page, et à chaque position, crée une forme et la dessine. Pour commencer, les formes sont des ronds.



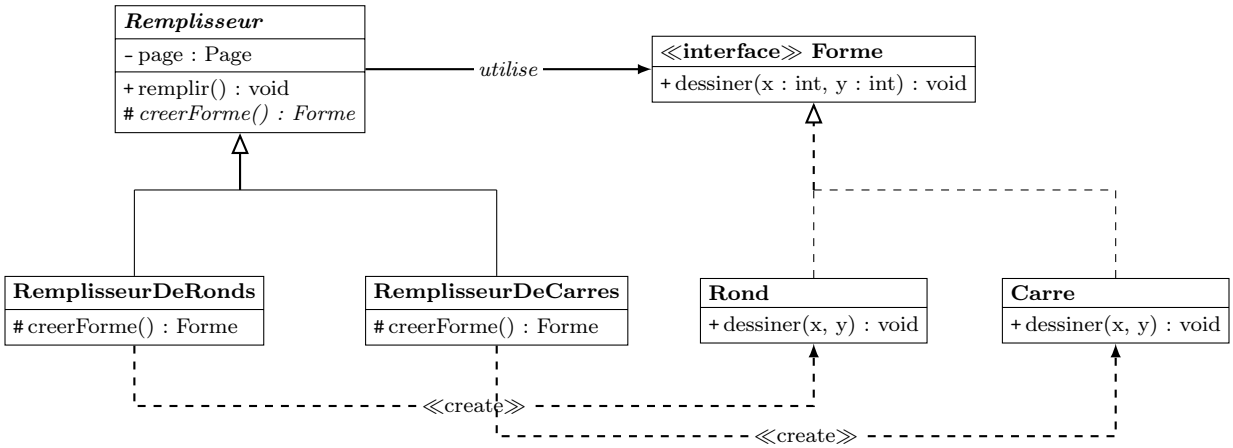
Le corps de `remplir` contient quelque part l'instruction `new Rond()`. Ce diagramme pose deux problèmes.

— Pour remplir une page de carrés, il faut modifier `Remplisseur`, alors que la façon de parcourir la page, elle, ne change pas (principe ouvert/fermé).

— `Remplisseur` dépend d'une classe concrète, alors qu'il faut privilégier les dépendances vers des types abstraits (principe d'inversion des dépendances).

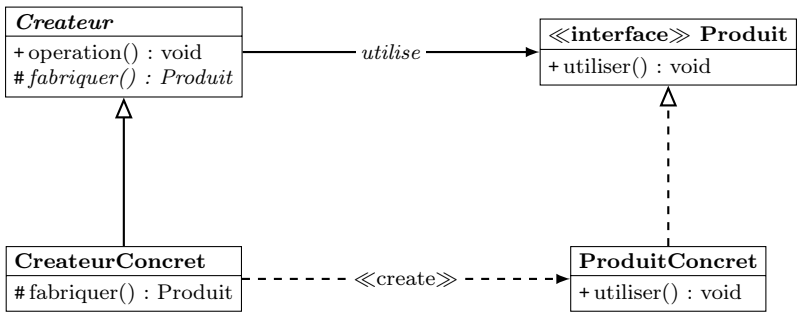
Pour les résoudre, il suffit de constater que `Remplisseur` fait deux choses de natures différentes : il parcourt la page, ce qui ne varie pas, et il crée une forme, ce qui varie. On sépare donc les

deux : la création devient une méthode à part, que `Remplisseur` déclare sans la définir, et que ses sous-classes définissent.



Ceci est un exemple d'utilisation du patron Fabrique.

Le schéma classique du patron Fabrique



Le patron fait intervenir quatre rôles.

- Le **produit** est l'abstraction de ce qui est créé. C'est le seul type que le créateur manipule.
- Le **produit concret** en est une réalisation.
- Le **créateur** contient le traitement qui a besoin d'un produit. Il déclare la méthode de fabrication sans la définir, et l'appelle dans ses propres méthodes.
- Le **créateur concret** définit la méthode de fabrication, et c'est le seul endroit du modèle où le nom d'un produit concret apparaît.

Le point essentiel est que le créateur appelle une méthode qu'il ne définit pas : il décrit ce qu'il faut faire du produit, sans jamais dire lequel. Choisir le produit revient alors à choisir le créateur concret, c'est-à-dire à changer une seule instruction, celle qui l'instancie.

À retenir

Les conditions d'utilisation du patron Fabrique.

- Une classe a besoin de créer des objets, mais la classe concrète à créer n'est pas la même selon les cas d'usage.
- Le traitement qui entoure cette création, lui, ne varie pas.
- Le choix du produit doit pouvoir être fait ailleurs que dans le code qui s'en sert.

Les avantages de ce patron.

- Ajouter un produit revient à écrire deux classes (le produit et son créateur) sans en modifier aucune (principe ouvert/fermé).
- Le code qui se sert du produit ne dépend que de son abstraction (principe d'inversion

des dépendances).

— Les noms des classes concrètes sont rassemblés en un seul endroit, au lieu d'être dispersés dans tous les `new` du programme.

— Changer de produit ne demande de modifier qu'une seule instruction.

Son prix.

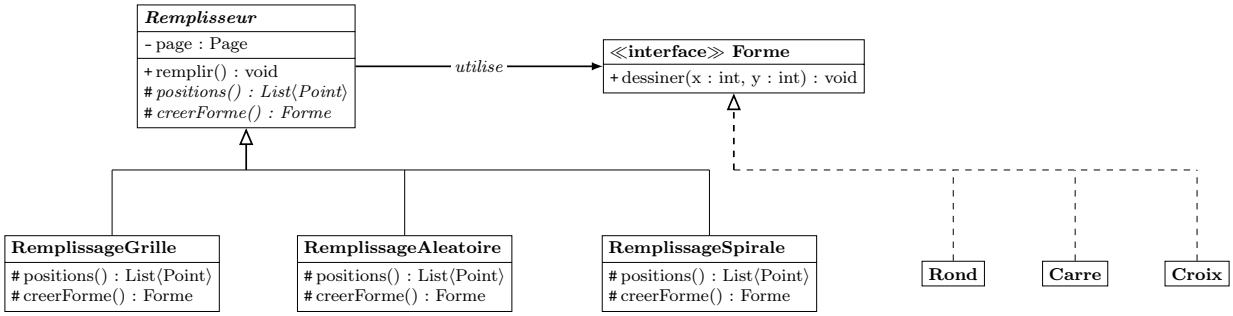
— Le nombre de classes double : un créateur concret par produit concret.

— Le choix du créateur doit bien être écrit quelque part : le patron déplace cette décision, il ne la supprime pas.

2.2 Application : remplir une page

Un utilisateur veut remplir une page A4 avec un assortiment de ronds, de carrés et de croix, sans que les formes ne se superposent. Il veut disposer de plusieurs méthodes de remplissage (en grille régulière, en disposition aléatoire, en spirale). Pour passer d'une méthode de remplissage à une autre, il ne doit être nécessaire de modifier qu'une seule ligne de code.

La page contient maintenant un assortiment : une même exécution crée des ronds, des carrés et des croix. Ce n'est donc plus le créateur concret qui fixe la forme une fois pour toutes, mais chaque appel à la méthode de fabrication qui en rend une. Ce qui varie d'un remplisseur à l'autre, ce sont les positions occupées, et le choix des formes qui les occupent.



La ligne unique à modifier est celle qui instancie le remplisseur, par exemple `Remplisseur r = new RemplissageGrille(page);`. Tout le reste du programme ne connaît que le type `Remplisseur` et ne voit aucune différence.

Ajouter un triangle revient à écrire une classe `Triangle` qui réalise `Forme`, et à la rendre atteignable par les méthodes de fabrication ; ajouter une quatrième méthode de remplissage revient à écrire une sous-classe de `Remplisseur`. Dans les deux cas, aucune classe existante n'est modifiée.

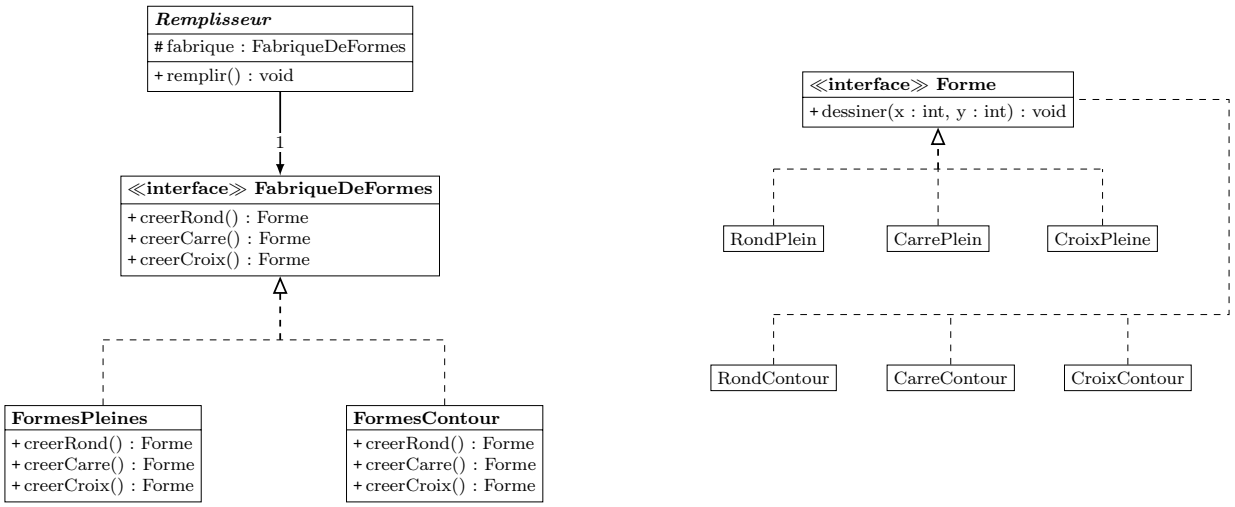
2.3 La fabrique abstraite

Le problème

L'éditeur du programme veut maintenant que les formes puissent être dessinées pleines ou en contour seul, et qu'une même page soit homogène : on ne mélange pas un rond plein et un carré en contour.

Il y a donc deux dimensions de variation : la sorte de forme (rond, carré, croix) et le style (plein, contour). Les traiter par héritage donnerait six classes, et dix avec une troisième sorte de forme. Surtout, rien ne garantirait l'homogénéité : une méthode de fabrication qui rend une `Forme` quelconque peut parfaitement rendre un rond plein puis un carré en contour.

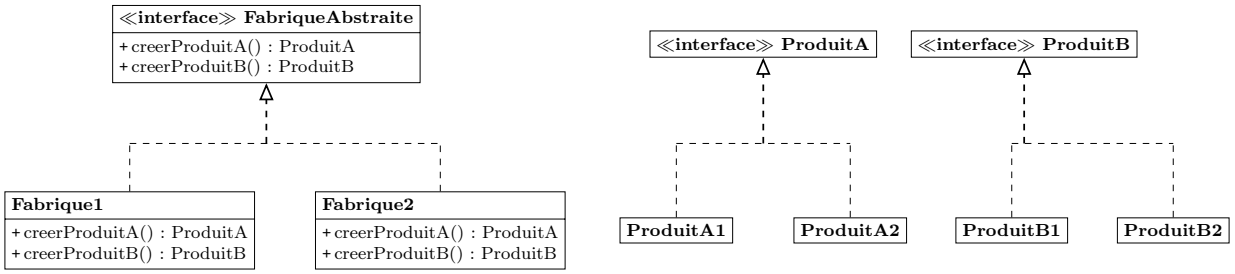
Pour le résoudre, on ne fabrique plus une forme à la fois, mais une *famille* de formes. La fabrique déclare une méthode par sorte de forme, et chaque fabrique concrète rend des formes du même style.



(Les dépendances de création ne sont pas tracées pour ne pas surcharger la figure : `FormesPleines` crée les trois formes pleines, `FormesContour` les trois formes en contour.)

Le remplisseur ne connaît que l'interface `FabriqueDeFormes` : il demande un rond, un carré ou une croix, et obtient des formes du même style sans jamais savoir lequel. L'homogénéité est garantie par construction, puisqu'une fabrique concrète ne sait produire qu'un style.

Le schéma classique de la Fabrique abstraite



Chaque fabrique concrète correspond à une *famille* de produits : `Fabrique1` rend `ProduitA1` et `ProduitB1`, `Fabrique2` rend `ProduitA2` et `ProduitB2`. Le client ne peut donc pas mélanger un produit d'une famille avec un produit d'une autre.

À retenir

Fabrique ou fabrique abstraite ?

— La Fabrique crée *un* produit, et c'est la sous-classe du créateur qui décide lequel.

— La Fabrique abstraite crée *une famille* de produits liés entre eux, et garantit que le client n'en mélange pas deux.

— On passe de l'une à l'autre dès qu'il y a plusieurs sortes de produits devant rester cohérentes entre elles.

Les avantages de la fabrique abstraite.

— La cohérence d'une famille est garantie par construction, et non vérifiée après coup.

— Ajouter une famille revient à écrire une fabrique concrète et ses produits, sans modifier aucune classe existante.

— Le client ne nomme aucun produit concret : il ne connaît que les interfaces et la fabrique qu'on lui a donnée.

Son prix.

— Ajouter une *sorte* de produit oblige à modifier l'interface de la fabrique et toutes ses

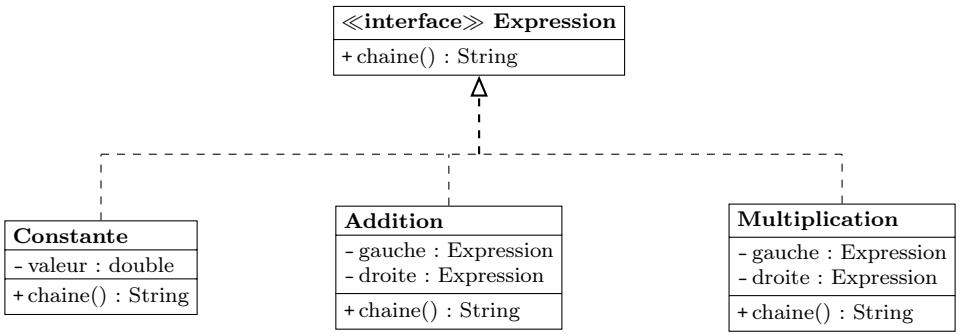
réalisations. Le patron est ouvert aux nouvelles familles, fermé aux nouvelles sortes.
— Le nombre de classes est le produit du nombre de sortes par le nombre de familles.

3 Le patron de méthode

Le problème

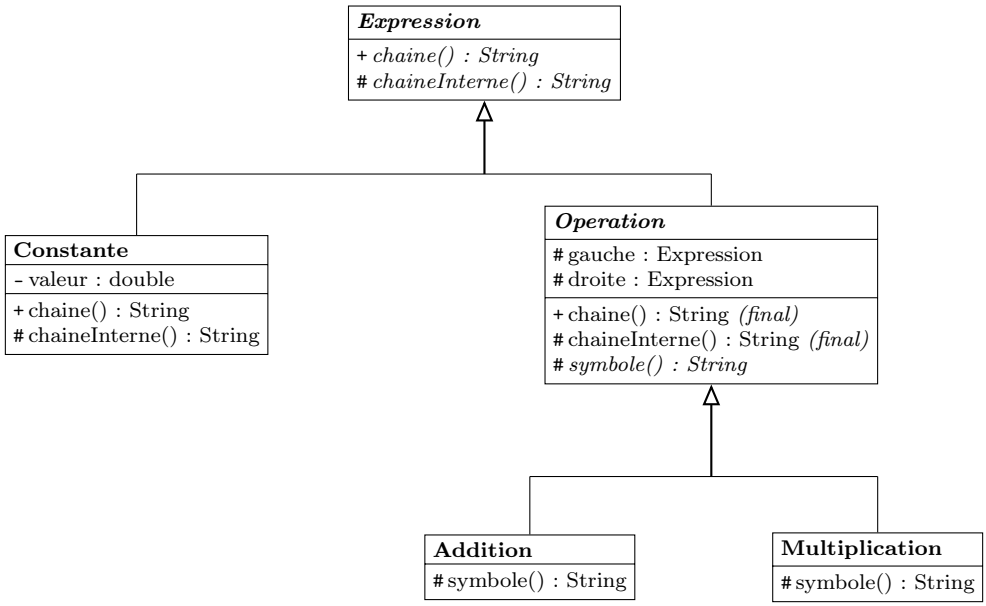
On appelle *opération* la donnée d'un symbole (+ ou ×) et de deux opérandes, chaque opérande étant soit une constante, soit elle-même une opération. On souhaite produire la chaîne de caractères représentant une opération, en parenthésant toute opération apparaissant comme opérande : ainsi l'addition de 2 et de la multiplication de 4 par 3 s'écrit `2+(4×3)`.

Chaque sorte d'opération sait s'écrire, ce qui donne le diagramme suivant.



Ce diagramme pose un problème : les méthodes `chaine` d'`Addition` et de `Multiplication` sont identiques à un caractère près. Toutes deux écrivent l'opérande gauche, puis un symbole, puis l'opérande droit, et parenthésent les sous-expressions. Ajouter la soustraction reviendrait à recopier ce code une troisième fois, et corriger une erreur de parenthésage obligerait à le faire autant de fois qu'il y a d'opérations.

Pour le résoudre, il suffit de constater que le déroulement de l'écriture est le même pour toutes les opérations, et que seul le symbole en dépend. On écrit donc ce déroulement une seule fois, dans une classe abstraite, en y laissant un trou : une méthode déclarée mais non définie, que chaque opération remplit.

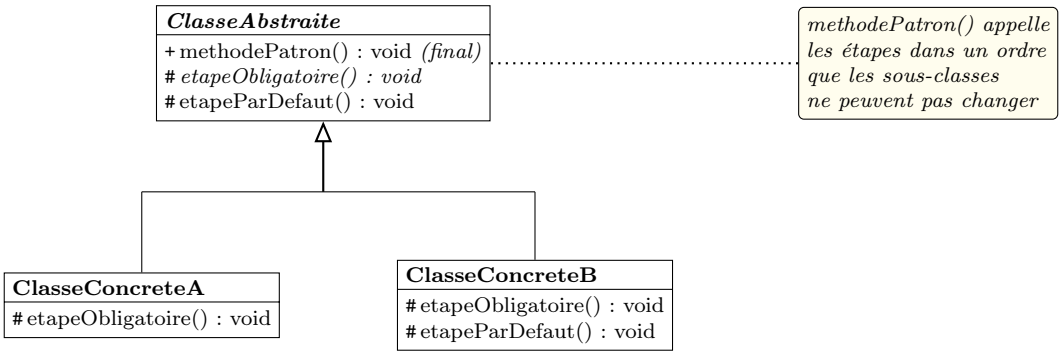


Il y a deux méthodes de production de chaîne. La méthode publique `chaine` est celle du premier appel : elle écrit l'expression sans la parenthéser. La méthode interne `chaineInterne` est celle qu'une opération appelle sur ses opérandes : elle entoure le résultat de parenthèses. Une constante n'en a pas besoin, donc ses deux méthodes rendent la même chose.

Tout l'algorithme tient ainsi dans `Operation` : `chaine` demande à l'opérande gauche sa chaîne interne, y ajoute le symbole, puis fait de même à droite. Le parenthésage ne demande aucun test de type, puisque c'est l'opérande lui-même qui sait s'il doit se parenthéser. `Addition` et `Multiplication` se réduisent alors à une ligne.

Ceci est un exemple d'utilisation du PATRON DE MÉTHODE.

Le schéma classique du PATRON DE MÉTHODE



Le patron fait intervenir trois éléments.

— La **méthode patron** contient le déroulement, c'est-à-dire la suite des étapes et leur ordre. Elle est déclarée `final` pour qu'une sous-classe ne puisse pas la redéfinir : c'est elle qui garantit que le déroulement est le même partout.

— Les **étapes obligatoires** sont déclarées sans être définies : chaque sous-classe doit les fournir.

— Les **étapes par défaut** ont une définition dans la classe abstraite, qu'une sous-classe peut remplacer si elle le souhaite.

Le point essentiel est l'inversion du sens des appels : ce n'est pas la sous-classe qui appelle la classe mère, c'est la classe mère qui appelle la sous-classe. La sous-classe ne décide pas de *quand* son code s'exécute, seulement de *ce qu'il fait*.

À retenir

Les conditions d'utilisation du PATRON DE MÉTHODE.

— Plusieurs classes font la même chose, dans le même ordre, et ne diffèrent que par quelques étapes.

— Le déroulement, lui, ne doit pas varier : il fait partie de ce qu'on veut garantir.

Les avantages de ce patron.

— Le déroulement est écrit une seule fois : le corriger le corrige partout.

— Une sous-classe se réduit à ce qui la distingue vraiment.

— Ajouter une variante revient à écrire une sous-classe sans modifier aucune classe existante (principe ouvert/fermé).

— Le compilateur signale les étapes obligatoires qu'on a oublié de définir.

Son prix.

— Le déroulement est figé : si une variante a besoin d'un ordre différent, le patron ne s'applique plus.

— Il repose sur l'héritage : une sous-classe est liée à sa classe mère pour toujours, et ne peut pas changer de déroulement en cours d'exécution.

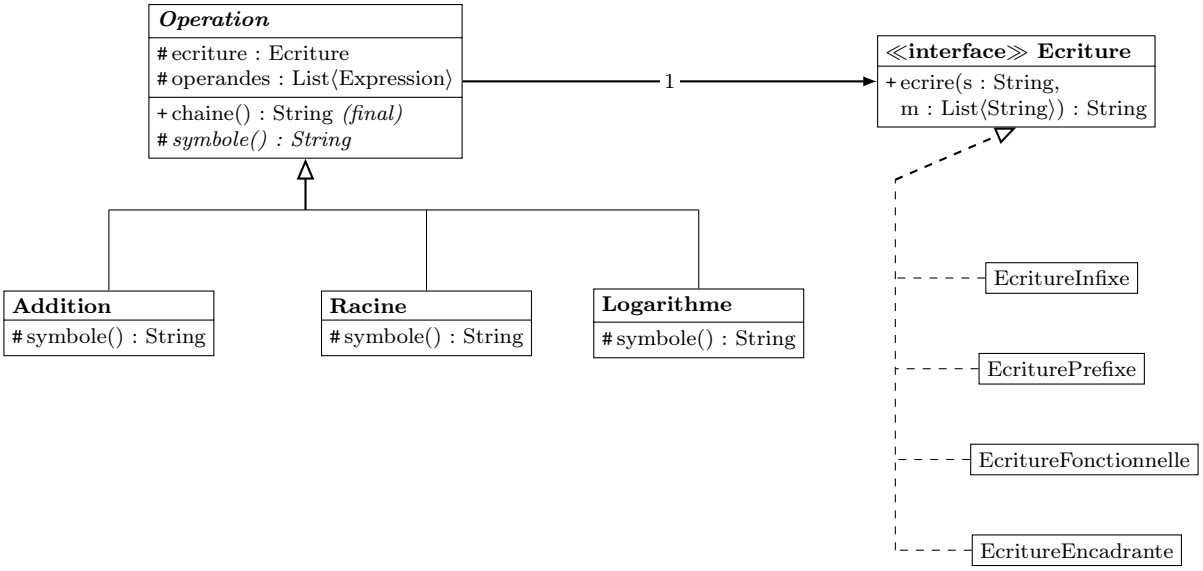
4 Le patron Stratégie

Le problème

On reprend les expressions, et on souhaite les enrichir d'opérations plus variées : la racine carrée, le logarithme, la valeur absolue. Mais ces opérations ne s'écrivent pas comme les précédentes : la racine carrée n'a qu'un opérande et s'écrit devant lui, $\sqrt{4+3}$; le logarithme s'écrit sous forme fonctionnelle, $\log(2, 8)$; la valeur absolue encadre son opérande, $|2+3|$.

Le déroulement de la section précédente (opérande gauche, symbole, opérande droit) ne convient donc plus à personne. Ce n'est plus une étape qui varie d'une opération à l'autre, c'est l'algorithme tout entier. On pourrait rendre `chaine` abstraite et la redéfinir dans chaque opération, mais on perdrait toute factorisation : la forme fonctionnelle serait réécrite autant de fois qu'il y a d'opérations fonctionnelles, et une opération ne pourrait jamais changer de façon de s'écrire, puisque son écriture serait soudée à sa classe.

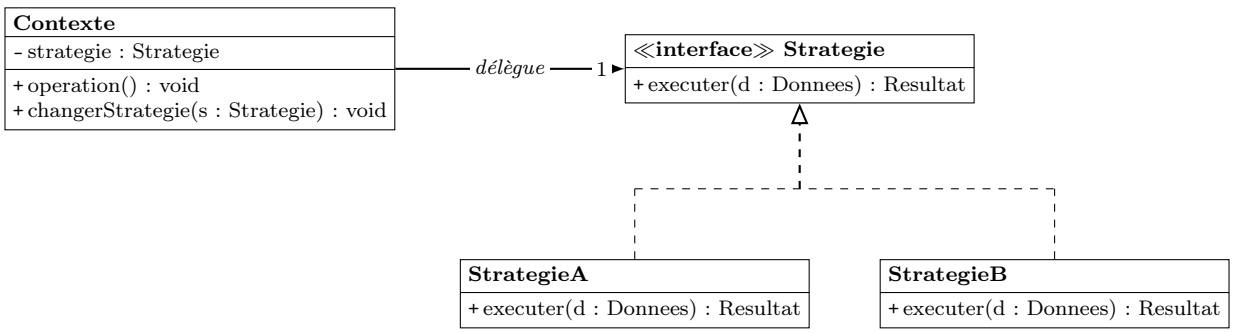
Pour le résoudre, il suffit de constater qu'il y a peu de *formes* d'écriture pour beaucoup d'opérations : infixe, préfixe, fonctionnelle, encadrante. On fait donc de la forme d'écriture un objet à part, que l'opération détient et auquel elle délègue.



La méthode `chaine` se réduit à : écrire chacun de ses opérandes, puis demander à son écriture d'assembler le tout à partir du symbole et de ces morceaux. `Addition` détient une `EcritureInfixe`, `Racine` une `EcriturePrefixe`, `Logarithme` une `EcritureFonctionnelle`. Aucune écriture ne connaît aucune opération : elle reçoit une chaîne et des morceaux, et rend une chaîne.

Ceci est un exemple d'utilisation du patron Stratégie.

Le schéma classique du patron Stratégie



Le patron fait intervenir trois rôles. La **stratégie** est l'interface de l'algorithme ; les **stratégies concrètes** en sont les réalisations ; le **contexte** détient une stratégie et lui délègue le travail, sans

jamais savoir laquelle il détient.

Le point essentiel est que l'algorithme est devenu un *objet*. On peut donc le passer en paramètre, le remplacer en cours d'exécution, et faire partager la même stratégie par plusieurs contextes.

À retenir

Patron de méthode ou stratégie ?

— Dans le PATRON DE MÉTHODE, le déroulement est fixe et seules certaines *étapes* varient. La variation passe par l'héritage, et elle est fixée une fois pour toutes à la construction de l'objet.

— Dans la STRATÉGIE, c'est l'*algorithme entier* qui varie. La variation passe par la composition, et peut changer en cours d'exécution.

— En une phrase : le patron de méthode remplit des trous dans un algorithme, la stratégie remplace l'algorithme.

Les avantages de la stratégie.

— L'algorithme peut être choisi, remplacé ou partagé à l'exécution.

— Ajouter un algorithme revient à écrire une classe, sans modifier aucune classe existante (principe ouvert/fermé).

— Le contexte n'a plus de branchement : il ne teste pas quel algorithme appliquer, il le détient.

— Chaque algorithme est isolé, donc lisible et testable séparément.

Son prix.

— Il faut bien que quelqu'un choisisse la stratégie : le patron déplace cette décision hors du contexte, il ne la supprime pas.

— L'interface de la stratégie doit convenir à toutes les réalisations : si l'une a besoin d'une donnée que les autres n'utilisent pas, elle figure quand même dans la signature.

— Une stratégie qui aurait besoin de savoir à quel type d'objet elle s'applique devrait le tester, ce qui enfreindrait le principe ouvert/fermé. C'est la limite du patron, et l'objet de la section suivante.

5 Le patron VISITEUR

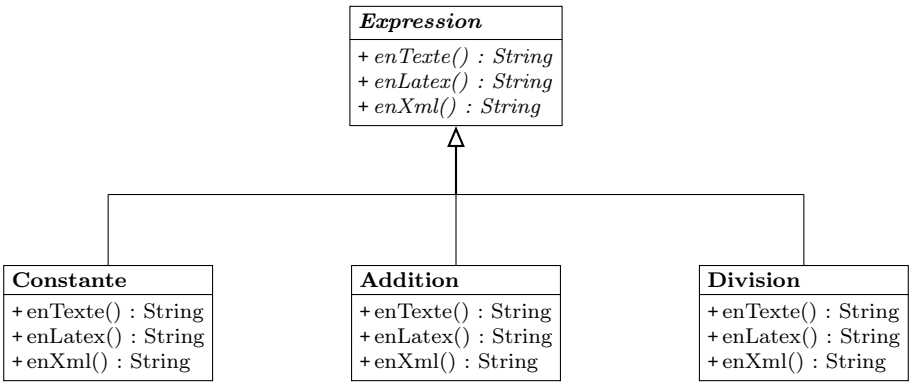
5.1 Les bases

Le problème

On reprend les expressions, et on veut maintenant en produire plusieurs représentations : du texte brut, du LaTeX, du XML. Ces représentations ne donnent pas le même symbole à une même opération, et ne lui donnent pas non plus la même forme. Ainsi, la division de 2 par 3 s'écrit `2/3` en texte brut, `\frac{2}{3}` en LaTeX, et `<div><cn>2</cn><cn>3</cn></div>` en XML.

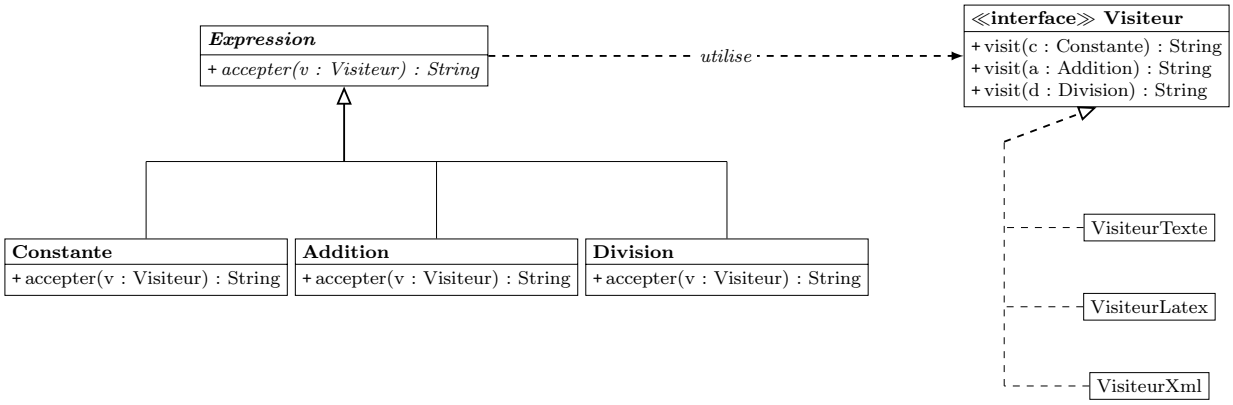
La stratégie de la section précédente ne suffit plus. Une écriture y recevait le symbole de l'opération et ses morceaux déjà écrits ; or le symbole dépend désormais du format, et la façon d'assembler les morceaux dépend à la fois du format et de l'opération. Il y a donc deux choses qui varient en même temps, et c'est leur *croisement* qu'il faut écrire.

La première idée est de donner une méthode par format à chaque expression.



Cela fonctionne, mais ajouter un format oblige à modifier toutes les classes d'expression, ce qui enfreint le principe ouvert/fermé. On pourrait aussi écrire une classe par format, qui recevrait une expression et la traiterait : il lui faudrait alors tester de quelle sorte d'expression il s'agit, ce qui enfreint le même principe.

Pour le résoudre, il suffit de constater que l'expression, elle, sait toujours de quelle sorte elle est. Au lieu de laisser le format le découvrir, c'est l'expression qui le lui dit : elle reçoit un objet représentant le format, et l'expression appelle sur lui la méthode qui lui correspond. Chacun des deux apporte ainsi la moitié de l'information.



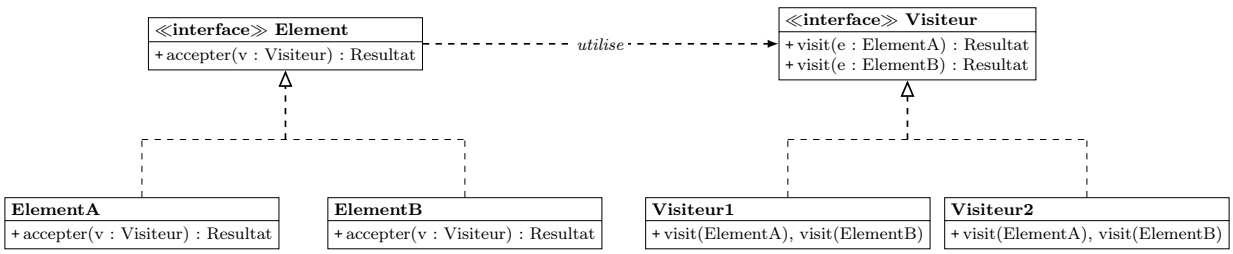
Les trois méthodes du visiteur portent le même nom : c'est le *type de leur paramètre* qui les distingue. Le corps de `accepter` tient alors en une ligne, la même partout : `return v.visit(this)`. Dans `Addition`, `this` est de type `Addition`, donc le compilateur choisit `visit(Addition)` ; dans `Division`, il choisit `visit(Division)`.

C'est pour cela que `accepter` doit être réécrit dans chaque classe, bien que son texte soit identique : si on l'écrivait une seule fois dans `Expression`, `this` y serait de type `Expression` et le compilateur ne saurait quelle surcharge appeler. Chaque classe d'expression n'a donc qu'une seule méthode, quel que soit le nombre de formats, et aucune classe ne comporte de test de type.

Ajouter un format revient alors à écrire une classe qui réalise `Visiteur`, sans modifier aucune classe existante.

Ceci est un exemple d'utilisation du patron Visiteur.

Le schéma classique du patron Visiteur



Le mécanisme s'appelle la **double délégation** : le client appelle `accepter` sur un élément sans savoir lequel, l'élément appelle `visit` sur le visiteur sans savoir lequel, et le traitement qui s'exécute est celui qui correspond au couple (élément, visiteur). Le premier appel est résolu à l'exécution, par le type réel de l'élément ; le second l'est à la compilation, par la surcharge. Aucun des deux n'a eu besoin d'un test de type.

À retenir

Les conditions d'utilisation du patron Visiteur.

— Deux choses varient en même temps : la sorte d'objet traité, et le traitement qu'on lui applique.

— On veut pouvoir ajouter des traitements, et l'ensemble des sortes d'objets, lui, est stable.

— Les objets traités acceptent d'exposer ce qu'il faut au traitement : le visiteur travaille de l'extérieur.

Les avantages de ce patron.

— Chaque classe visitée n'a qu'une seule méthode, `accepter`, quel que soit le nombre de traitements.

— Ajouter un traitement revient à écrire une classe, sans modifier aucune classe existante (principe ouvert/fermé).

— Un traitement est rassemblé en un seul endroit, au lieu d'être réparti dans toutes les classes visitées.

— Aucun test de type : le compilateur garantit qu'un visiteur traite bien toutes les sortes d'éléments.

Son prix.

— Ajouter une *sorte* d'élément oblige à modifier l'interface du visiteur et toutes ses réalisations. Le patron est ouvert aux nouveaux traitements, fermé aux nouvelles sortes : c'est exactement l'inverse de ce qu'on obtient en mettant les traitements dans les classes visitées.

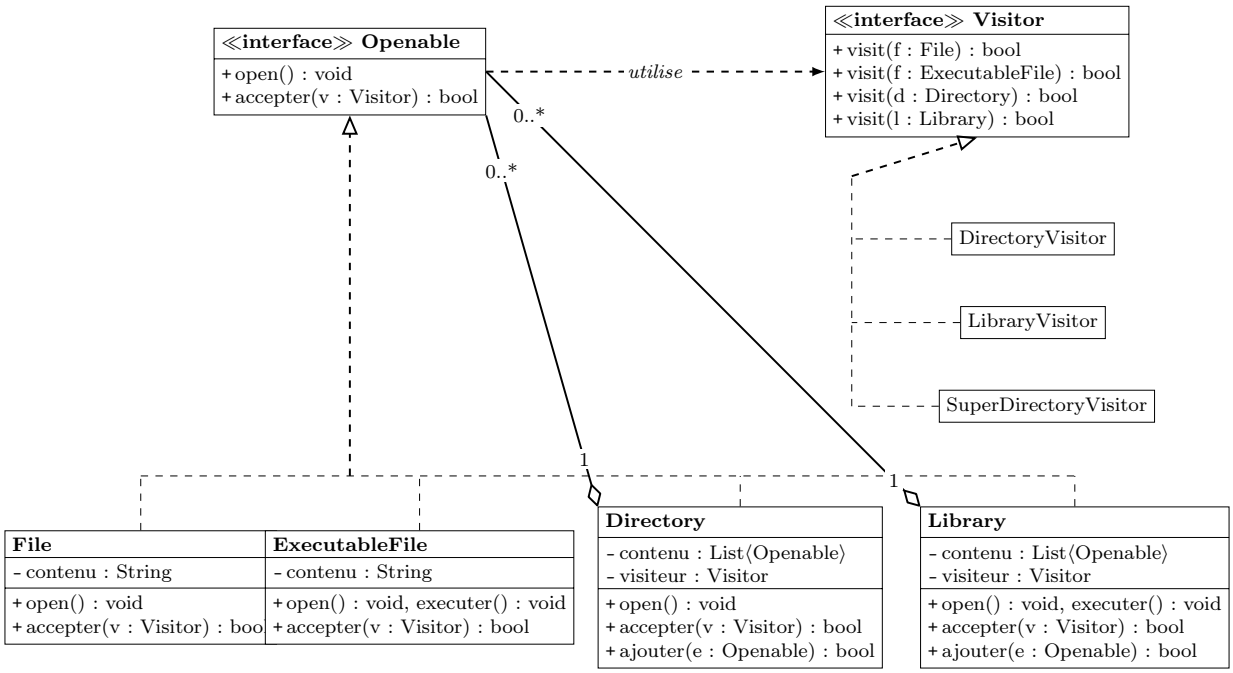
— Le visiteur agit de l'extérieur : il ne voit des éléments que ce qu'ils rendent public, ce qui peut obliger à ouvrir leur encapsulation.

— Les deux hiérarchies se connaissent : le visiteur nomme toutes les sortes d'éléments.

5.2 Application : retour sur le système de fichiers

Reprenons le système de fichiers de la section 1.3. Les règles d'ajout y étaient portées par le typage : trois interfaces, et autant de classes de conteneurs qu'il y avait de politiques d'ajout. Le visiteur permet d'en sortir ces règles, et de les traiter comme un traitement parmi d'autres.

L'idée est qu'un conteneur ne décide plus lui-même de ce qu'il accepte : il détient un visiteur, à qui il demande si l'élément candidat lui convient. La méthode `ajouter` se réduit alors à `si (e.accepter(visiteur)) alors contenu.ajouter(e)`, et c'est le visiteur qui porte la politique.



(Les associations de `Directory` et de `Library` vers `Visitor` ne sont pas tracées : elles se lisent dans l'attribut `visiteur`.)

Il n'y a plus qu'une seule classe de dossier. En section 1.3, `Directory` et `SuperDirectory` ne différaient que par ce qu'ils acceptaient comme enfant ; cette décision étant maintenant prise au-dehors, par le visiteur, les deux classes n'ont plus aucune différence de rôle et se confondent. Un dossier ordinaire est un `Directory` muni d'un `DirectoryVisitor`, un super-dossier est le même `Directory` muni d'un `SuperDirectoryVisitor`. `Library`, en revanche, reste une classe à part : elle ne se distingue pas par sa politique d'ajout mais par le fait qu'elle s'exécute.

Les interfaces `Executable` et `Conteneur` disparaissent elles aussi : elles ne servaient qu'à typer les listes, et les listes sont désormais toutes des listes d'`Openable`.

Ce que l'on gagne, ce que l'on perd

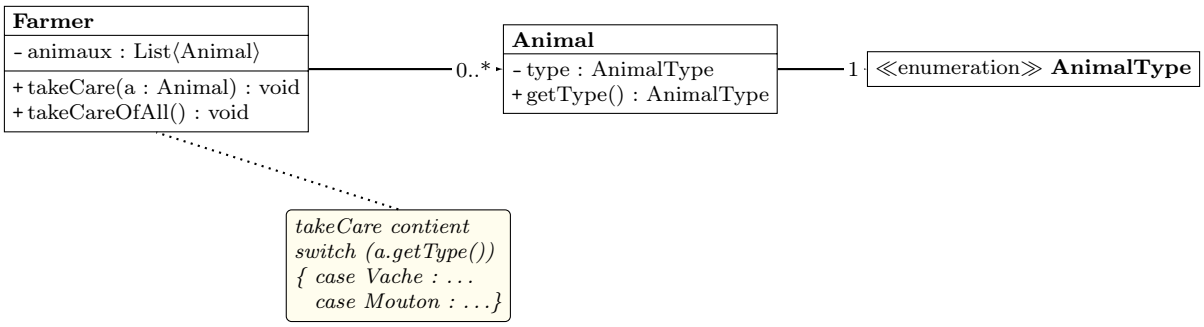
Par rapport à la section 1.3, le modèle est plus petit : une classe de conteneur au lieu de deux, et plus d'interfaces de contenu. Surtout, la politique d'ajout est devenue une donnée : un même dossier peut en changer en cours d'exécution, ce qu'aucune construction par typage ne permet. En revanche, la garantie change de nature. En section 1.3, ajouter un dossier à une bibliothèque ne compilait pas ; ici, cela compile et rend `false`. On est passé d'une vérification à la compilation à une vérification à l'exécution. C'est le prix de la souplesse, et le choix entre les deux dépend de ce que l'on préfère garantir.

6 Le patron Chaîne de responsabilité

Le problème

Un fermier possède plusieurs types d'animaux, et chaque animal requiert un soin particulier. On impose la structure suivante : les animaux sont représentés par une *unique* classe `Animal`, dont le type est donné par un attribut de type `AnimalType`, une énumération dont le contenu évoluera ; le fermier est une unique classe `Farmer`, qui détient la liste de ses animaux et possède une fonction `takeCare(a : Animal)`.

La première idée est d'écrire ce choix dans le fermier, sous la forme d'un branchement sur le type de l'animal.



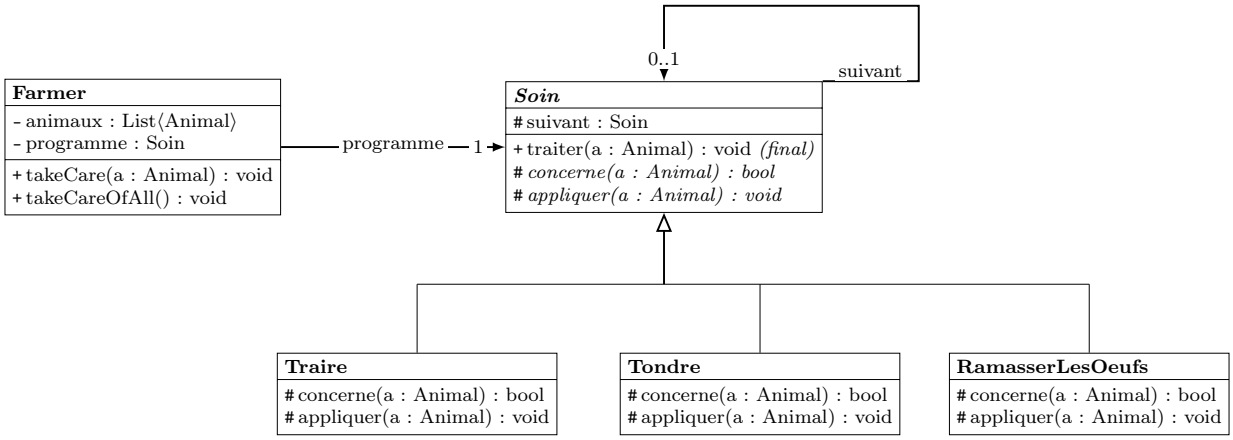
Ce diagramme pose deux problèmes.

— Ajouter un type d'animal oblige à modifier `Farmer`, alors que le reste de son travail ne change pas (principe ouvert/fermé). Un `switch` n'est acceptable que sur un ensemble dont le nombre d'éléments est immuable, ce qui n'est pas le cas ici.

— `Farmer` connaît tous les soins et la façon de les choisir : il fait deux choses à la fois.

On pourrait remplacer le branchement par une table associant chaque type d'animal à son soin, mais la difficulté reste la même : cette table est une classe de plus qu'il faut rouvrir à chaque nouveau type d'animal.

Pour le résoudre, il suffit de constater que chaque soin sait de lui-même s'il concerne un animal donné. Au lieu de demander à quelqu'un de choisir, on présente l'animal au premier soin : s'il le concerne, il s'en charge ; sinon, il le passe au suivant.

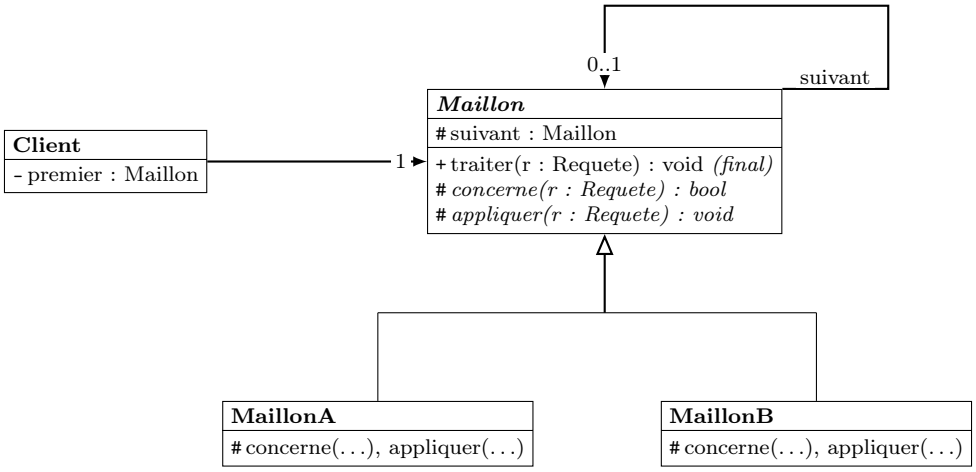


La méthode `traiter` contient tout le mécanisme, et une seule fois : si le soin concerne l'animal, il l'applique ; sinon, il passe la main à son suivant, s'il en a un. Le fermier ne connaît plus qu'un seul soin, le premier de la chaîne, et ignore tout des autres.

Le branchement n'a pas disparu, il a été éclaté : chaque soin porte sa propre condition, et ajouter un type d'animal revient à écrire un soin de plus et à l'insérer dans la chaîne, sans modifier aucune classe existante.

Ceci est un exemple d'utilisation du patron Chaîne de responsabilité.

Le schéma classique du patron Chaîne de responsabilité



Le point essentiel est que le client ne connaît que le premier maillon. Il ne sait ni combien ils sont, ni lequel traitera la requête, ni même si l'un d'eux la traitera. L'ordre des maillons fait partie des règles : présentée à une chaîne autrement assemblée, la même requête peut recevoir un autre traitement.

À retenir

Les conditions d'utilisation de la Chaîne de responsabilité.

- Plusieurs traitements sont candidats pour une même demande, et c'est à eux de décider s'ils sont concernés.
- Celui qui émet la demande n'a pas à savoir qui va s'en charger.
- L'ordre dans lequel les candidats sont interrogés a un sens, et doit pouvoir être décidé ailleurs que dans leur code.

Les avantages de ce patron.

- Le branchement central disparaît : chaque maillon ne porte que sa propre condition.
- Ajouter un traitement revient à écrire une classe et à l'insérer dans la chaîne, sans modifier aucune classe existante (principe ouvert/fermé).
- L'émetteur et les traitements sont découplés : l'émetteur ne connaît que le premier maillon.
- La composition et l'ordre de la chaîne sont des données, modifiables à l'exécution.

Son prix.

- Rien ne garantit qu'une demande soit traitée : il faut prévoir un maillon terminal, ou accepter qu'elle ressorte sans réponse.
- Le parcours a un coût, et le comportement dépend d'un ordre qui ne se lit nulle part dans les classes : le diagramme ne dit pas ce que la chaîne fera.
- Suivre l'exécution est plus difficile : on ne sait pas, à la lecture, quel maillon a traité quoi.

7 Stratégie, visiteur et chaîne : trois façons de déléguer

Ces trois patrons se ressemblent : dans les trois cas, un objet confie un traitement à un autre objet qu'il ne connaît que par une interface. Ils répondent pourtant à trois questions différentes.

- La Stratégie répond à « *comment* faire ? » Le contexte sait ce qu'il veut, et délègue la manière de le faire à un objet unique, qui fait tout le travail.

— Le Visiteur répond à « *que* faire selon la sorte d'objet ? » Le traitement dépend d'un couple, et c'est le croisement des deux hiérarchies qu'il faut écrire.

— La Chaîne de responsabilité répond à « *qui* s'en occupe ? » Plusieurs traitements sont candidats, et on cherche le premier qui se déclare.

	Stratégie	**Visiteur**	**Chaîne**
Objets sollicités	un seul	un seul	plusieurs, jusqu'au premier qui traite
Qui décide du traitement	celui qui fournit la stratégie	le couple (élément, visiteur)	chaque maillon, pour lui-même
L'ordre compte	non	non	oui
Dépend du type de l'objet traité	non	oui	oui
Ajouter un traitement	une classe	une classe	une classe
Ajouter une sorte d'objet traité	rien	tous les visiteurs	un maillon
Garantie	le traitement a lieu	le compilateur vérifie l'exhaustivité	aucune : la demande peut ressortir

Les deux dernières lignes sont celles qui tranchent le plus souvent. Si l'ensemble des sortes d'objets est stable et qu'on veut la garantie que tous les cas sont traités, c'est le visiteur. Si cet ensemble s'enrichit sans cesse, le visiteur devient coûteux et la chaîne lui est préférable, au prix de la garantie. Et si le traitement ne dépend pas de la sorte de l'objet, mais seulement d'une décision prise ailleurs, la stratégie suffit, et c'est la plus simple des trois.

Un cas limite

Une chaîne dont tous les maillons traitent la demande sans jamais l'arrêter n'est plus une chaîne de responsabilité : on ne cherche plus un responsable, on diffuse une information à des abonnés. L'ordre devient alors indifférent, et c'est un autre patron, l'Observateur, qu'il faut employer.